



Arquitectura de agentes basados en modelos de lenguaje

Fundamentos, marcos de desarrollo y un caso práctico con LangGraph y Ollama

MUIA
Máster Universitario en Inteligencia Artificial

Escuela Superior de Informática
Universidad de Castilla-La Mancha
Ciudad Real

5 de octubre de 2026

Documento académico. Toda la bibliografía ha sido verificada contra las fuentes primarias y todos los listados de resultados proceden de ejecuciones reales.

Resumen

Los modelos de lenguaje de gran escala han dejado de ser generadores de texto para convertirse en el núcleo de sistemas que actúan sobre el mundo. La literatura identifica cuatro elementos constitutivos en la arquitectura de estos agentes —perfil, memoria, herramientas y planificación—, pero existe una brecha entre esa caracterización académica y los materiales con los que se enseña, porque las interfaces de alto nivel de los marcos de desarrollo ocultan precisamente los mecanismos que se pretende explicar. Este documento sintetiza los fundamentos académicos de la arquitectura de agentes basados en modelos de lenguaje, revisa el entorno LangChain y su evolución hacia LangGraph, analiza las capacidades que un formalismo de grafo de estados aporta al desarrollo de agentes y caracteriza Ollama como vía de acceso a modelos, en local y en la nube. Sobre esa base se presenta un taller intensivo como caso práctico, diseñado de forma incremental de modo que cada elemento arquitectónico corresponda a una modificación observable del grafo. La verificación experimental, realizada sobre un entorno real, confirma que el llamado a funciones es la capacidad crítica del sistema, que la salida estructurada solo es fiable cuando se impone por el canal de funciones y que la volatilidad de los catálogos de modelos exige verificar la disponibilidad antes de cada sesión. Se concluye que la elección del mecanismo de orquestación es una decisión didáctica además de técnica, y que los fallos silenciosos constituyen el contenido de mayor valor formativo.

Palabras clave: agentes basados en LLM, arquitecturas de agentes, LangGraph, LangChain, Ollama, ReAct, llamado a funciones, memoria, planificación.

Índice

1. Introducción **4**
1.1. Motivación 4
1.2. Objetivos y contribuciones 4
1.3. Estructura del documento 5

2. Fundamentos: de los modelos de lenguaje a los agentes **5**
2.1. La arquitectura *transformer* y el aprendizaje en contexto 5
2.2. Ajuste por instrucciones 5
2.3. Modelos aumentados y el salto al agente 6
2.4. Definición operativa de agente basado en LLM 6

3. Elementos de la arquitectura de un agente basado en LLM **7**
3.1. El perfil: identidad, restricciones y contrato de formato 8
3.2. La memoria: continuidad frente a conocimiento 8
3.3. Las herramientas: extender las capacidades más allá del texto 9

3.4. La planificación: descomposición y control . . . 10
3.5. El bucle percepción–acción y el papel del orquestador . . . 12

4. LangChain: composición de aplicaciones con modelos de lenguaje 12
4.1. Origen y filosofía de diseño . . . 12
4.2. El lenguaje de composición de expresiones . . . 13
4.3. Salida estructurada . . . 13
4.4. Memoria conversacional . . . 13
4.5. La limitación estructural: el bucle como caja negra . . . 14

5. LangGraph: agentes como grafos de estado 14
5.1. El modelo computacional . . . 14
5.2. El bucle ReAct como estructura explícita . . . 15
5.3. Persistencia y memoria . . . 17
5.4. Interrupciones y supervisión humana . . . 18
5.5. Transmisión incremental y observabilidad . . . 19
5.6. Patrones multiagente . . . 19
5.7. Comparación con planteamientos alternativos . . . 20

6. Acceso a los modelos: Ollama 20
6.1. La elección del proveedor como decisión arquitectónica . . . 20
6.2. Despliegue local . . . 21
6.3. El servicio en la nube . . . 22
6.4. Compatibilidad con la API de OpenAI y llamado a funciones . . . 22
6.5. Reproducibilidad: la caducidad de los catálogos . . . 23

7. Caso práctico: el taller 24
7.1. Del fundamento al artefacto docente . . . 24
7.2. Los cuatro módulos . . . 24
7.3. Metodología de verificación . . . 26
7.4. Resultados . . . 26
7.5. Discusión didáctica . . . 27

8. Discusión 28
8.1. Limitaciones . . . 28
8.2. Riesgos y seguridad . . . 28
8.3. Buenas prácticas . . . 29

9. Conclusiones y trabajo futuro 29

Referencias 30

1 Introducción

1.1 Motivación

Los modelos de lenguaje de gran escala (LLM, *Large Language Model*) han pasado en pocos años de ser sistemas de generación de texto a convertirse en el componente central de aplicaciones que **actúan** sobre el mundo: consultan bases de datos, invocan servicios, escriben y ejecutan código, y mantienen interacciones prolongadas con usuarios humanos. A este tipo de sistemas se les denomina **agentes basados en LLM**, y su estudio ha generado un cuerpo de literatura que se ha consolidado en apenas tres años [1], [2], [3].

La literatura coincide en una observación que resulta especialmente relevante desde el punto de vista docente: la capacidad de un LLM para resolver una tarea de forma autónoma no depende únicamente del modelo, sino de la **arquitectura** que lo rodea. Un mismo modelo, con los mismos pesos, puede comportarse como un generador de texto plausible o como un agente capaz de resolver un problema en varios pasos, según cómo se le proporcione memoria, herramientas y un mecanismo de planificación [4], [5]. La diferencia es de ingeniería, no de escala.

Sin embargo, existe una asimetría notable entre la literatura y la práctica docente. Los trabajos académicos describen arquitecturas a un nivel de abstracción que no se traduce directamente en código, mientras que la documentación de los marcos de desarrollo —la fuente que consultan los estudiantes— presenta interfaces de alto nivel que **ocultan precisamente los mecanismos que se pretenden enseñar**. El resultado es un patrón de fallo bien conocido: el estudiante consigue ejecutar un agente de diez líneas, pero cuando ese agente alucina una respuesta o ignora una herramienta, no dispone de un modelo mental que le permita localizar el problema.

Este trabajo parte de esa tensión y la aborda con una decisión metodológica concreta: **utilizar un marco en el que la estructura del agente sea un objeto explícito, inspeccionable y modificable**, en lugar de un artefacto interno de la biblioteca. LangGraph [6] permite representar un agente como un grafo de estados en el que cada elemento arquitectónico —el perfil, las herramientas, la memoria y la planificación— ocupa una posición visible y verificable. Esa propiedad, que a primera vista parece un detalle de implementación, tiene consecuencias didácticas profundas.

1.2 Objetivos y contribuciones

El objetivo de este documento es doble. En primer lugar, **sintetizar los fundamentos académicos** de la arquitectura de agentes basados en LLM, identificando los elementos que la literatura considera constitutivos y relacionándolos con las técnicas que los implementan. En segundo lugar, **trasladar esa síntesis a un artefacto docente verificable**: un taller intensivo cuyos materiales —libretas ejecutables, transparencias y guiones— han sido ejecutados y comprobados sobre una infraestructura real de acceso a modelos.

Las contribuciones concretas que se presentan son las siguientes.

1. **Una arquitectura de referencia de cuatro elementos** (perfil, memoria, herramientas y planificación) articulada sobre el bucle percepción–acción, con la correspondencia explícita entre cada elemento, su formulación académica y su realización en un grafo de estados.
2. **Una revisión del entorno LangChain y de su evolución hacia LangGraph**, identificando la limitación estructural de los flujos acíclicos que motiva el cambio de paradigma hacia los grafos con ciclos.
3. **Un análisis de las capacidades que LangGraph aporta** al desarrollo de agentes: estado tipado con reducers, aristas condicionales, *checkpointing* con memoria por hilo, interrupciones para la supervisión humana, transmisión incremental y patrones multiagente.

4. **Una caracterización de Ollama como vía de acceso a modelos** en sus dos modalidades, local y en la nube, con sus implicaciones de reproducibilidad: se documenta empíricamente el fenómeno de la **caducidad de los catálogos de modelos**, que afecta directamente a la validez de los materiales docentes.
5. **Un caso práctico completo**, el taller, con su diseño instruccional incremental y la evidencia de ejecución recogida durante su verificación.

1.3 Estructura del documento

La Sección 2 establece los fundamentos: qué es un agente basado en LLM y cómo se ha llegado hasta ahí desde la arquitectura *transformer* y el ajuste por instrucciones. La Sección 3 desarrolla los cuatro elementos de la arquitectura. La Sección 4 introduce el entorno LangChain y sus límites. La Sección 5 se centra en LangGraph y en las capacidades que proporciona. La Sección 6 presenta Ollama como método de acceso a un modelo, en local y en la nube. La Sección 7 describe el taller como caso práctico y recoge la evidencia experimental. La Sección 8 discute limitaciones, riesgos y buenas prácticas, y la Sección 9 cierra con las conclusiones.

2 Fundamentos: de los modelos de lenguaje a los agentes

2.1 La arquitectura *transformer* y el aprendizaje en contexto

El sustrato de todo el edificio es la arquitectura *transformer* [7], basada en mecanismos de atención que permiten modelar dependencias entre posiciones de una secuencia sin recurrencia. Sobre esa arquitectura se construyeron los modelos de lenguaje autorregresivos que definen el estado del arte actual, y cuyo comportamiento emergente quedó descrito con claridad en el trabajo sobre GPT-3 [8]: un modelo entrenado para predecir el siguiente token puede resolver tareas para las que no fue entrenado explícitamente con solo recibir unos pocos ejemplos en el contexto.

Ese fenómeno, el **aprendizaje en contexto** (*in-context learning*), es la condición de posibilidad de los agentes modernos. Significa que el comportamiento del modelo puede modificarse sin tocar sus pesos, únicamente alterando el texto que recibe. Un agente, por tanto, es en buena medida **un programa escrito en lenguaje natural y ejecutado por un intérprete estocástico**.

Las limitaciones de ese sustrato son igualmente conocidas y condicionan el diseño de las arquitecturas. Un LLM puro es, en lo esencial, una función sin estado —cada invocación es independiente—, sin acceso al mundo exterior más allá de su contexto, con una ventana de contexto finita y sin capacidad de verificar sus propias afirmaciones [9]. Cada uno de los cuatro elementos arquitectónicos que se describen en la Sección 3 puede entenderse como una respuesta a una de esas limitaciones.

2.2 Ajuste por instrucciones

El segundo ingrediente histórico es el **ajuste por instrucciones**. Los modelos entrenados únicamente con predicción de siguiente token son difíciles de dirigir; el ajuste fino supervisado sobre demostraciones y, sobre todo, el aprendizaje por refuerzo a partir de retroalimentación humana (RLHF) introducido en InstructGPT [10], producen modelos que siguen instrucciones de forma fiable. El trabajo posterior sobre ajuste con instrucciones a escala [11] confirmó que esta capacidad mejora con el número de tareas de ajuste y se transfiere a tareas no vistas.

Esta etapa es la que hace viable el concepto de **rol**: si el modelo no obedeciese instrucciones, un *prompt* de sistema que declarase «eres un nutricionista y respondes con cifras» no tendría

efecto sistemático. La fiabilidad del seguimiento de instrucciones es, pues, el presupuesto sobre el que se apoya el primer elemento de la arquitectura.

2.3 Modelos aumentados y el salto al agente

La literatura distingue entre **modelos de lenguaje aumentados**, que combinan el modelo con mecanismos externos de recuperación, herramientas y razonamiento, y **agentes**, que añaden a lo anterior la capacidad de decidir autónomamente la secuencia de acciones [4]. La frontera no es nítida, pero el criterio operativo es útil: en un sistema aumentado el flujo lo decide el programador; en un agente, lo decide el modelo en tiempo de ejecución.

La propuesta de sistemas MRKL [12] anticipó con claridad el patrón que hoy domina: un **enrutador** que descompone la petición y delega cada parte en el módulo adecuado, sea un modelo de lenguaje o una herramienta simbólica. Esa idea de enrutamiento modular reaparece, generalizada, en los mecanismos de *function calling* que se describen en la Sección 3.3.

Un tercer antecedente conceptual es la arquitectura cognitiva **CoALA** [5], que traslada el vocabulario de la ciencia cognitiva a los agentes de lenguaje y propone distinguir entre memoria de trabajo, memoria episódica, memoria semántica y memoria procedimental, además de separar los procesos de decisión y de aprendizaje. Esta taxonomía es la que subyace, en buena medida, a la distinción entre memoria a corto y a largo plazo que se emplea en la Sección 3.2.

En paralelo, la analogía del **sistema operativo** ha resultado productiva para describir la pila completa: el LLM como núcleo, el contexto como memoria principal y los agentes como aplicaciones que compiten por un recurso escaso [13]. La utilidad de la analogía es doble: explica la gestión del contexto como un problema de asignación de recursos, y justifica la existencia de una capa de orquestación con estado, que es precisamente el papel que desempeña LangGraph en este trabajo.

2.4 Definición operativa de agente basado en LLM

Las encuestas del área ofrecen definiciones complementarias. Wang y colaboradores describen al agente autónomo como un sistema con **perfil**, **memoria**, **planificación** y **acción** [1]; Xi y colaboradores organizan el campo en torno a las capacidades de percepción, cerebro y acción [2]; Cheng y colaboradores revisan definiciones, métodos y perspectivas e insisten en la naturaleza iterativa del comportamiento agéntico [3]; y Masterman y colaboradores comparan las arquitecturas emergentes atendiendo a razonamiento, planificación e invocación de herramientas [14].

De este conjunto se adopta aquí la siguiente definición operativa, que es la que estructura el resto del documento:

> Un **agente basado en LLM** es un sistema que, disponiendo de un **perfil** que fija su identidad y sus restricciones, de una **memoria** que le permite mantener continuidad, de un conjunto de **herramientas** que extienden sus capacidades más allá del texto y de un mecanismo de **planificación** que organiza sus acciones, ejecuta de forma iterativa un bucle de percepción y acción con el objetivo de satisfacer una petición expresada en lenguaje natural.

La definición tiene la virtud de ser **verificable**: cada uno de los cuatro elementos se corresponde con una construcción concreta en el marco de desarrollo, y su ausencia produce un modo de fallo característico y observable. La Tabla 1 resume esa correspondencia, que se desarrolla en la Sección 3 y se materializa en el caso práctico de la Sección 7.

Tabla 1: Tabla 1. Los cuatro elementos de la arquitectura y el modo de fallo asociado a su ausencia. Elaboración propia a partir de [1], [2], [5].

Elemento	Limitación del LLM que resuelve	Modo de fallo si falta
Perfil	Falta de especialización y de restricciones	Respuestas genéricas, formato inestable
Memoria	Ausencia de estado entre invocaciones	Amnesia conversacional
Herramientas	Desconexión del mundo y del cálculo	Alucinación de datos verificables
Planificación	Ejecución en un único paso	Bucles, pasos repetidos, objetivos no alcanzados

3 Elementos de la arquitectura de un agente basado en LLM

Esta sección desarrolla los cuatro elementos de la definición operativa de la Sección 2.4. El orden no es arbitrario: reproduce la **dependencia funcional** entre ellos. El perfil determina *quién* decide; la memoria, *con qué información*; las herramientas, *qué puede hacer*; y la planificación, *en qué orden*. La Figura 1 presenta la arquitectura de referencia completa, con el bucle percepción–acción que las integra.

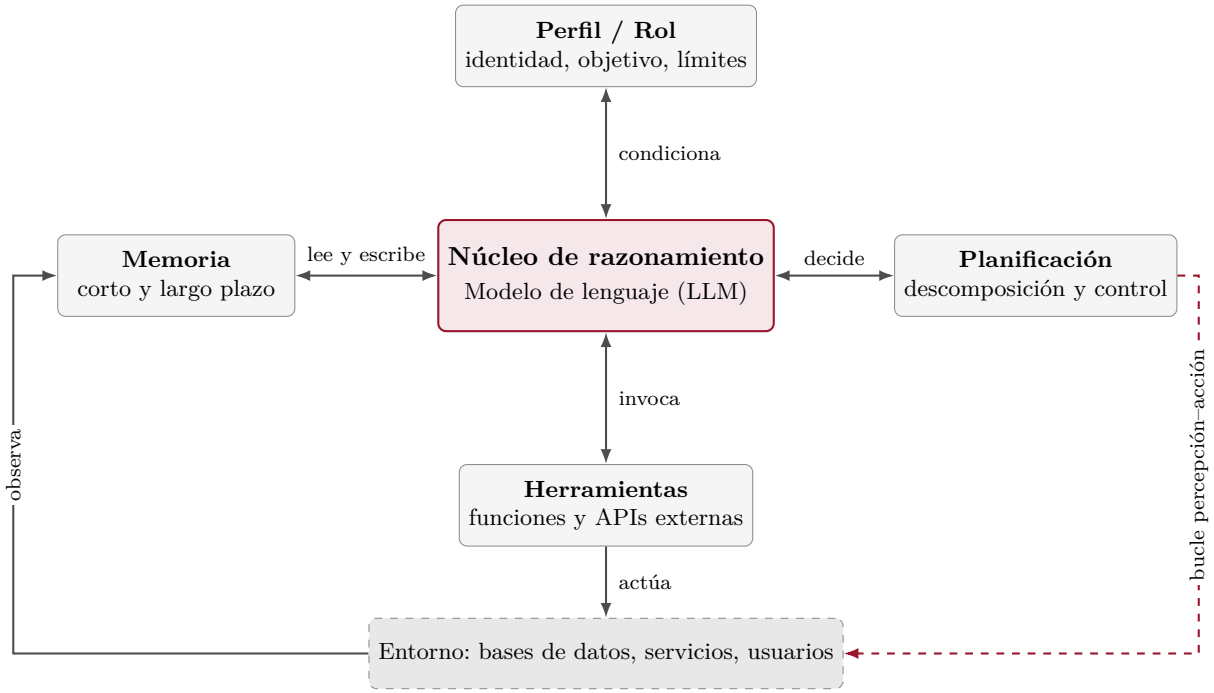


Figura 1: Arquitectura de referencia de un agente basado en LLM. El núcleo de razonamiento recibe el perfil que condiciona su comportamiento, intercambia información con la memoria, decide apoyándose en el mecanismo de planificación e invoca herramientas que actúan sobre el entorno; las observaciones resultantes realimentan el ciclo. Elaboración propia a partir de [@wang2023survey; @sumers2023cognitive; @mialon2023augmented].

3.1 El perfil: identidad, restricciones y contrato de formato

El **perfil** —denominado *profile* en [1] y *persona* en buena parte de la literatura de *prompting*— es el conjunto de instrucciones que fija la identidad del agente, su objetivo y sus límites. Técnicamente se materializa en el mensaje de sistema que precede a la conversación y que, por tanto, el modelo recibe en cada invocación. Su eficacia sistemática presupone la capacidad de seguir instrucciones descrita en la Sección 2.2 [10].

Conviene distinguir con precisión dos funciones que el perfil cumple y que con frecuencia se confunden.

La primera es **orientar el contenido**: el vocabulario, el enfoque, el nivel de detalle y las prioridades de la respuesta. Un mismo modelo, ante una misma petición, produce respuestas funcionalmente distintas según el perfil declarado. Esta función es real pero **no determinista**: el perfil sesga la distribución de salidas, no la restringe.

La segunda es **fijar el formato**. Aquí la literatura y la práctica divergen de forma instructiva. Pedir en el mensaje de sistema que la respuesta sea un objeto JSON con un esquema dado es una instrucción más, sujeta a la variabilidad estocástica del modelo. La alternativa robusta consiste en **imponer el formato por un canal distinto del texto**: la generación guiada por gramáticas o esquemas [15] restringe el espacio de secuencias decodificables de modo que la salida es válida por construcción, no por obediencia.

Esa distinción tiene una consecuencia práctica de primer orden para el diseño de agentes: **el formato no se solicita, se impone**. Y el mecanismo para imponerlo es, en la mayoría de los marcos actuales, el mismo que se emplea para invocar herramientas: la definición de una función cuyo esquema de argumentos es el formato deseado. La Sección 3.3 desarrolla ese mecanismo, y el caso práctico de la Sección 7.2 lo explota explícitamente para mostrar al alumnado que la salida estructurada y las herramientas son, en el fondo, la misma construcción.

Desde el punto de vista docente, esta es una de las ideas que más rentabilidad tiene, porque explica un síntoma que de otro modo resulta desconcertante: un agente que «a veces» devuelve JSON válido y «a veces» no. No es un comportamiento errático del modelo; es la diferencia entre sugerir y restringir.

3.2 La memoria: continuidad frente a conocimiento

Un LLM es, por construcción, **sin estado**: cada invocación es independiente y el modelo no conserva nada entre llamadas. Todo comportamiento que un usuario percibe como «memoria» es el resultado de reenviar información en cada invocación. Esta observación, trivial en apariencia, es la clave para entender por qué la memoria es un componente arquitectónico y no una propiedad del modelo.

La taxonomía más influyente en este ámbito es la de CoALA [5], que distingue entre memoria de trabajo —la información activa durante la resolución de una tarea—, memoria episódica —experiencias concretas—, memoria semántica —hechos— y memoria procedimental —habilidades y rutinas, encarnadas en los pesos del modelo o en su código de apoyo. En la práctica de los agentes basados en LLM, esa distinción se simplifica en dos categorías operativas, que la Figura 2 representa.

La **memoria a corto plazo** es la conversación en curso. Su soporte físico es la ventana de contexto, y su problema característico es que esa ventana es finita mientras que el historial crece de forma monótona. Las estrategias de gestión son tres: el recorte de los turnos más antiguos, el resumen periódico del historial y la recuperación selectiva de los fragmentos relevantes. La literatura sobre el mecanismo de memoria en agentes de lenguaje sistematiza estas alternativas y sus compromisos [16]. Conviene subrayar que el recorte y el resumen no son optimizaciones opcionales: son condiciones para que el sistema siga funcionando, y su ausencia produce degradación

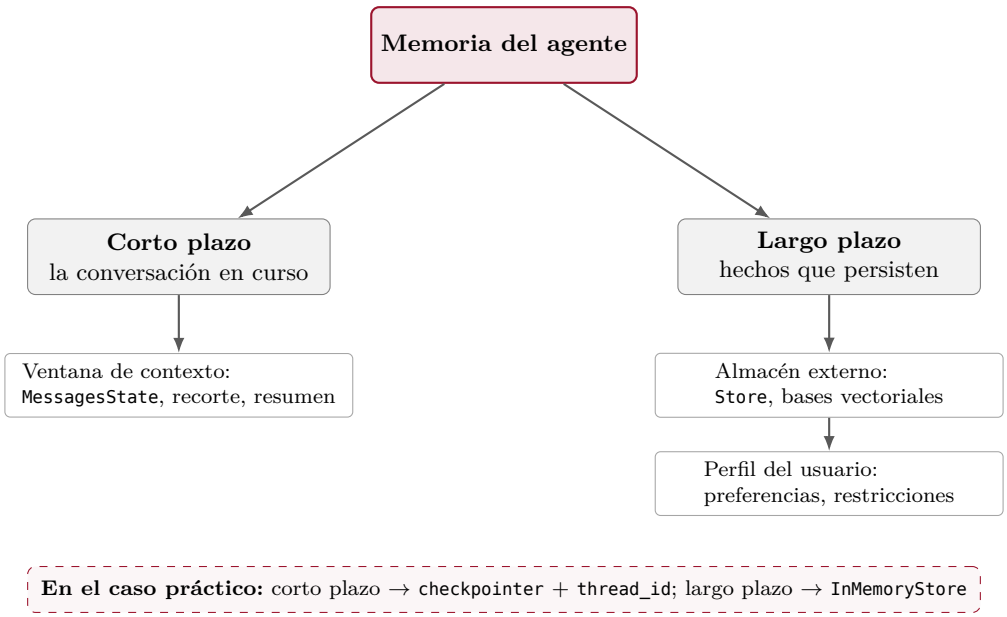


Figura 2: Taxonomía operativa de la memoria de un agente y su correspondencia con los mecanismos empleados en el caso práctico. La memoria a corto plazo resuelve la continuidad dentro de una conversación; la memoria a largo plazo conserva información que debe sobrevivir a conversaciones distintas. Elaboración propia a partir de [@sumers2023cognitive; @park2023generative; @zhang2024memory].

progresiva de la latencia y del coste.

La **memoria a largo plazo** conserva información que no pertenece a una conversación concreta: preferencias del usuario, hechos aprendidos, resultados de tareas anteriores. Su soporte es un almacén externo, y su problema característico es la **recuperación**: en qué momento y con qué criterio se incorpora un recuerdo al contexto. Dos trabajos resultan especialmente ilustrativos. *Generative Agents* [17] demuestra que un registro de experiencias con recuperación por relevancia, recencia e importancia produce comportamientos coherentes a largo plazo en agentes simulados. MemGPT [18] lleva la analogía del sistema operativo hasta sus últimas consecuencias y trata el contexto como memoria principal gestionada explícitamente por el propio modelo mediante llamadas de paginación.

La distinción entre ambas memorias no es meramente conceptual: se identifican por claves distintas y, en consecuencia, se implementan con mecanismos distintos. La memoria a corto plazo se indexa por **conversación**; la memoria a largo plazo, por **usuario**. Confundir ambos ejes es una fuente clásica de errores y se aborda explícitamente en la Sección 5.3.

3.3 Las herramientas: extender las capacidades más allá del texto

El tercer elemento responde a una limitación evidente: un LLM no puede consultar una base de datos, ejecutar un cálculo fiable ni actuar sobre un servicio externo. Las **herramientas** son funciones externas que el agente puede invocar, y su incorporación es una de las líneas de trabajo más activas del área.

El antecedente directo es Toolformer [19], que muestra que un modelo puede aprender de forma auto-supervisada *cuándo* conviene llamar a una API y *con qué* argumentos. El trabajo sobre aprendizaje de herramientas a partir de modelos fundacionales [20] sistematiza el problema en sus tres fases —demostración, entrenamiento y uso—; Gorilla [21] aborda la recuperación de

la API correcta en catálogos masivos, y ToolLLM [22] extiende el problema a más de dieciséis mil APIs del mundo real. La encuesta sobre modelos de lenguaje aumentados [4] enmarca todos estos resultados en una misma pregunta: cómo dotar al modelo de acceso verificable a información y capacidades que no posee.

En la práctica de los marcos actuales, el mecanismo de integración es el **llamado a funciones** (*function calling*), anticipado conceptualmente por la propuesta MRKL [12]. El principio es sencillo y conviene enunciarlo con precisión porque constituye el núcleo del módulo correspondiente del taller:

> El modelo **no ejecuta** la herramienta. Emite una petición estructurada —el nombre de la función y sus argumentos— y es el **orquestador** quien la ejecuta y devuelve el resultado al modelo como una observación.

Esta separación de responsabilidades tiene tres consecuencias de diseño. Primera: el control de lo que el agente puede hacer reside en el orquestador, que decide qué herramientas expone; el modelo no puede invocar nada que no se le haya ofrecido. Segunda: la ejecución es determinista y auditable, lo que permite registrar cada invocación. Y tercera: los errores de las herramientas son **información** para el agente, no excepciones fatales; una herramienta que falla puede devolver el error como observación y permitir que el modelo reaccione.

El contrato entre el modelo y la herramienta es su **esquema**, derivado del nombre, la firma y la documentación de la función. Esa documentación no es un comentario para humanos: es el texto a partir del cual el modelo decide cuándo usar la herramienta y cómo construir sus argumentos. Una descripción ambigua produce un uso errático, lo que constituye un modo de fallo específico y didácticamente muy aprovechable.

Finalmente, la trazabilidad de las invocaciones proporciona un criterio de verificación que resulta esencial en el aula. Si la traza de una ejecución no contiene ninguna invocación de herramienta, entonces el modelo **no consultó** ninguna fuente externa: cualquier dato cuantitativo de su respuesta proviene de su distribución de entrenamiento. Distinguir ambos casos es la diferencia entre un agente que consulta y un generador de texto plausible, y es una habilidad que el taller entrena explícitamente mediante la lectura de trazas.

3.4 La planificación: descomposición y control

El cuarto elemento aborda la pregunta de cómo organizar las acciones. La literatura ha propuesto tres familias de estrategias, resumidas en la Figura 3.

La base de todas ellas es la **cadena de pensamiento** (*chain-of-thought*), que muestra que solicitar al modelo un razonamiento intermedio mejora su desempeño en tareas que requieren varios pasos [23]. La autoconsistencia [24] refuerza esa idea mediante el muestreo de múltiples razonamientos y la votación sobre sus resultados. La generalización de estas técnicas a estructuras ramificadas da lugar a *Tree of Thoughts* [25] y *Graph of Thoughts* [26], que exploran y agregan alternativas de razonamiento a costa de un mayor consumo de cómputo.

La contribución decisiva para los agentes es **ReAct** [27], cuya formulación alterna el razonamiento en lenguaje natural con la ejecución de acciones y la incorporación de las observaciones resultantes. La Figura 4 representa ese ciclo. Su importancia es doble: por un lado, permite que el modelo corrija su rumbo con información del entorno; por otro, hace que el proceso sea **legible**, porque cada paso de razonamiento queda registrado junto a la acción que lo acompaña.

ReAct es, sin embargo, **reactivo**: decide un paso cada vez y no dispone de una visión global de la tarea. Para tareas largas esa ausencia de plan explícito se manifiesta en pasos redundantes o en bucles. La estrategia **plan-and-execute** [28] aborda esa limitación separando la descomposición de la ejecución: primero se elabora un plan explícito y después se ejecutan sus pasos. La ventaja

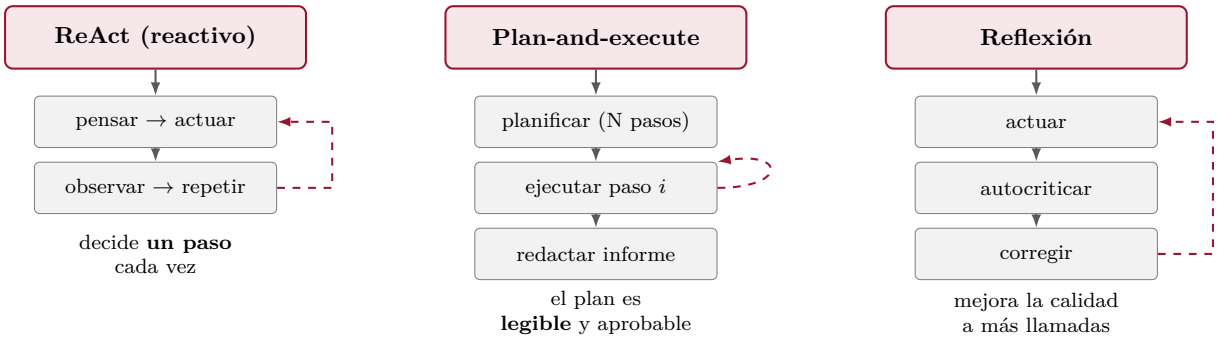


Figura 3: Tres familias de estrategias de planificación. ReAct decide un paso cada vez a partir de la observación anterior; plan-and-execute elabora un plan completo antes de actuar, lo que lo hace inspeccionable y aprobable; la reflexión introduce un ciclo de autocrítica sobre el resultado. Elaboración propia a partir de [@yao2022react; @wang2023plan; @shinn2023reflexion].

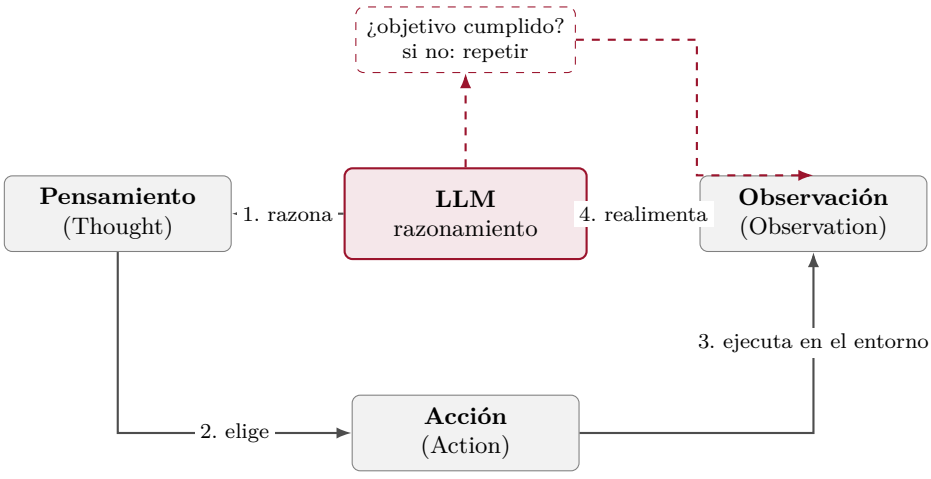


Figura 4: El ciclo ReAct. El modelo razona, selecciona una acción, el orquestador la ejecuta en el entorno y la observación resultante se incorpora al contexto, tras lo cual se evalúa la condición de parada. Elaboración propia a partir de [@yao2022react].

no es solo de eficiencia; un plan explícito es un objeto **inspeccionable y aprobable** antes de que se produzca cualquier efecto en el mundo, lo que lo convierte en el punto natural de inserción de la supervisión humana.

La tercera familia es la **reflexión**. Reflexion [29] introduce un ciclo en el que el agente evalúa su propio desempeño y almacena esa evaluación para mejorar en intentos sucesivos, mientras que Self-Refine [30] obtiene mejoras iterativas mediante autocrítica sin entrenamiento adicional. En el ámbito multiagente, la discusión entre varios modelos [31] muestra que el contraste entre perspectivas mejora la veracidad y el razonamiento. El coste común de esta familia es un mayor número de invocaciones al modelo, lo que introduce un compromiso explícito entre calidad y consumo de recursos.

3.5 El bucle percepción–acción y el papel del orquestador

Los cuatro elementos anteriores no operan de forma independiente: se integran en un **bucle** que da lugar al comportamiento agéntico (Figura 1). El componente que gobierna ese bucle —al que en este trabajo se denomina **orquestador**— es responsable de la secuencia de ejecución, de la gestión del estado, de la invocación de las herramientas y de las condiciones de terminación.

La elección del orquestador es, por tanto, una decisión arquitectónica de primer orden, y es precisamente el objeto de las tres secciones siguientes. Antes de abordarla conviene registrar dos observaciones que la literatura respalda.

La primera es que el comportamiento agéntico debe **evaluarse como tal**. AgentBench [32] establece un marco de evaluación multi-entorno que mide la capacidad de los modelos para actuar como agentes, y -bench [33] introduce dominios realistas con interacción entre agente, herramienta y usuario, subrayando que la evaluación de un agente no puede reducirse a la evaluación de sus respuestas finales.

La segunda es que la autonomía introduce **riesgos específicos** que condicionan el diseño. El trabajo sobre inyección indirecta de *prompts* [34] demuestra que un agente que consume contenido externo puede ser manipulado a través de ese contenido, lo que convierte la supervisión humana en un requisito y no en una opción. Las guías de interacción humano–IA de Amershi y colaboradores [35] proporcionan un marco consolidado para diseñar esos puntos de control, que en el caso práctico se materializan mediante interrupciones explícitas del grafo.

4 LangChain: composición de aplicaciones con modelos de lenguaje

4.1 Origen y filosofía de diseño

LangChain [36] es un marco de desarrollo cuyo propósito es proporcionar abstracciones comunes para construir aplicaciones basadas en modelos de lenguaje. Su planteamiento original parte de una observación pragmática: la mayoría de las aplicaciones con LLM repiten las mismas operaciones —construir un *prompt* a partir de una plantilla, invocar un modelo, analizar su salida, recuperar documentos y encadenar esos pasos— y, sin embargo, cada proveedor expone interfaces incompatibles.

La respuesta del marco es una **capa de abstracción de proveedor** y un **modelo de composición**. El primero permite sustituir el modelo subyacente sin reescribir la aplicación, lo que en el contexto de este trabajo resulta esencial: el mismo código funciona contra un modelo servido localmente o contra un servicio en la nube. El segundo es el que interesa aquí, porque determina qué arquitecturas de agente son expresables.

4.2 El lenguaje de composición de expresiones

El mecanismo de composición de LangChain se denomina **LCEL** (*LangChain Expression Language*) y se apoya en el operador de tubería. La idea es que los componentes se comportan como funciones componibles: una plantilla seguida de un modelo seguido de un analizador de salida produce una cadena.

```
cadena = plantilla | modelo | analizador
resultado = cadena.invoke({"ingrediente": "pollo", "minutos": 15})
```

Bajo esa sintaxis hay un contrato de interfaz —el protocolo *Runnable*— que garantiza que cualquier componente expone operaciones homogéneas de invocación, invocación por lotes y transmisión incremental. Esa uniformidad es valiosa en la práctica: permite que un componente compuesto se comporte como un componente simple y pueda anidarse.

Las piezas que la literatura de agentes identifica como elementos arquitectónicos tienen en LCEL una correspondencia directa. Las **plantillas de prompt** parametrizan el perfil; los **parciales** permiten fijar partes de la plantilla y dejar otras abiertas; los **mensajes de tipo sistema, humano y asistente** estructuran la conversación; y los **analizadores de salida** convierten texto en estructuras tipadas.

4.3 Salida estructurada

La conversión de la salida del modelo en un objeto tipado es uno de los puntos donde el marco aporta más valor y donde conviene ser preciso. LangChain ofrece una operación que, dado un modelo de datos, devuelve instancias validadas de ese modelo. Internamente existen varias estrategias de implementación, y la elección entre ellas no es indiferente.

La estrategia basada en el **canal de funciones** traduce el modelo de datos a un esquema de herramienta y obliga al modelo a rellenarlo; es la que emplea el mismo mecanismo que el llamado a funciones y, por tanto, la más robusta cuando el proveedor soporta ese canal. La estrategia basada en **esquema JSON** delega en el proveedor la restricción de la decodificación, y solo funciona si el servicio la implementa. Existe además una modalidad de **modo JSON** que se limita a solicitar texto con sintaxis JSON, sin garantía estructural alguna.

Esta distinción tiene consecuencias empíricas que se documentan en la Sección 7.4 y que constituyen una de las lecciones centrales del caso práctico: la elección del canal determina si la salida estructurada funciona de forma fiable o falla de manera intermitente.

4.4 Memoria conversacional

LangChain proporciona un mecanismo de memoria por sesión que añade historial a cualquier cadena compuesta. El patrón consiste en incluir en la plantilla un marcador de posición para el historial, definir una función que devuelva el historial correspondiente a un identificador de sesión y envolver la cadena con esa función. La invocación se realiza pasando el identificador en la configuración.

El mecanismo resuelve el problema de la continuidad conversacional descrito en la Sección 3.2 y lo hace de forma desacoplada de la lógica. Sin embargo, arrastra dos limitaciones que la práctica revela con rapidez. La primera es que la gestión del crecimiento del historial queda en manos del desarrollador. La segunda, más profunda, es que este mecanismo pertenece al paradigma **acíclico**: enriquece una cadena lineal con información adicional, pero no proporciona un lugar donde el control pueda volver atrás y decidir de nuevo.

4.5 La limitación estructural: el bucle como caja negra

La construcción de agentes en LangChain se apoyó inicialmente en un ejecutor de agentes que encapsulaba el bucle de razonamiento y acción. El programador declaraba un modelo, un conjunto de herramientas y una estrategia de decisión, y el ejecutor gestionaba la iteración internamente.

Esta aproximación tiene indudables ventajas de concisión, pero presenta tres problemas que la literatura y la práctica han ido haciendo evidentes.

El primero es de **transparencia**. El bucle —el elemento que materializa la planificación— no es visible ni modificable. Insertar un paso de validación, limitar el número de iteraciones con un criterio propio o añadir una aprobación humana exige conocer detalles internos del ejecutor.

El segundo es de **composición**: un flujo acíclico no puede expresar de forma natural los ciclos que caracterizan a los agentes. La Figura 5 contrasta ambos modelos. Una cadena es un grafo dirigido acíclico en el que cada paso ocurre una sola vez; un agente, en cambio, requiere volver a decidir a la luz de las observaciones, lo que introduce ciclos.

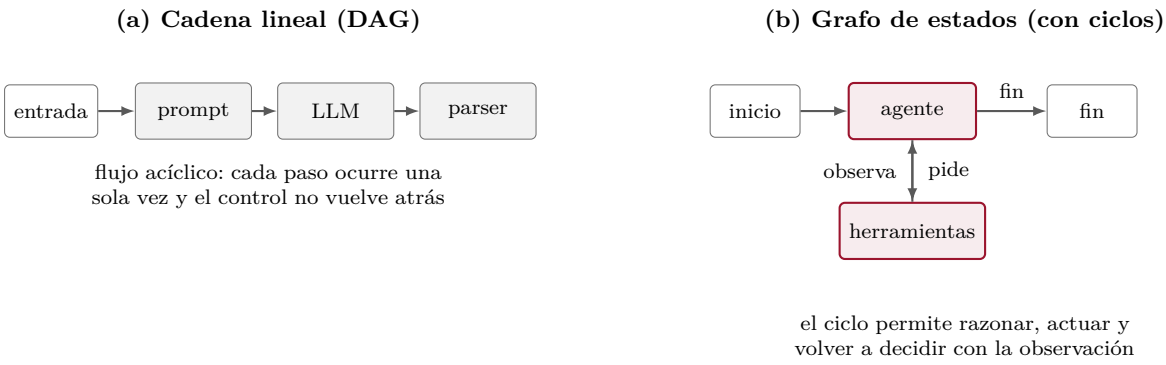


Figura 5: Dos modelos de composición. (a) La cadena lineal es un grafo acíclico: el control avanza sin retorno. (b) El grafo de estados admite ciclos, que es la estructura necesaria para que un agente razone, actúe y vuelva a decidir con la observación obtenida. Elaboración propia.

El tercero es de **sostenibilidad de la interfaz**. La propia documentación del marco señala que las construcciones clásicas de agentes han quedado obsoletas en favor de un enfoque basado en grafos, y las versiones recientes emiten avisos de obsolescencia que remiten explícitamente a la alternativa. Este detalle, que en un contexto profesional es una cuestión de mantenimiento, en un contexto docente se convierte en un criterio de selección: **no tiene sentido enseñar una interfaz cuya propia documentación recomienda sustituir**.

5 LangGraph: agentes como grafos de estado

5.1 El modelo computacional

LangGraph [6] traslada el problema a un modelo computacional explícito: un **grafo de estados** dirigido, potencialmente cíclico, en el que los nodos son unidades de cómputo y las aristas determinan el flujo de control. La Figura 6 resume su anatomía.

Los cuatro conceptos del modelo son los siguientes.

El **estado** es una estructura tipada —habitualmente un diccionario con anotaciones de tipo— cuyas claves constituyen los canales de información del grafo. El estado no es un detalle de implementación: es la interfaz entre los nodos, y su diseño determina qué información puede fluir.

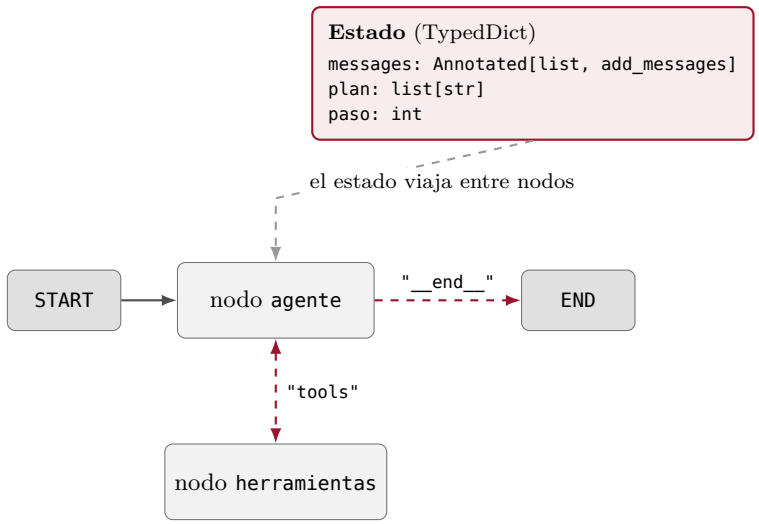


Figura 6: Anatomía de un grafo de estados. El estado es un diccionario tipado que viaja entre los nodos; cada nodo devuelve únicamente las claves que modifica y un *reducer* las fusiona con el estado anterior; las aristas condicionales traducen el resultado de una función de decisión en el siguiente nodo a ejecutar. Elaboración propia.

Los **nodos** son funciones que reciben el estado y devuelven un **parche**: únicamente las claves que modifican. Esta decisión de diseño merece atención porque es una fuente frecuente de errores en la práctica y, a la vez, una excelente ilustración del modelo. El nodo declara qué cambia, no reconstruye el estado completo.

Los **reducers** especifican cómo se combina el parche con el estado anterior. El caso por defecto es la sustitución del valor, pero los canales que acumulan información —el historial de mensajes, por ejemplo— se anotan con una función de agregación. La elección del reducer es una decisión semántica con consecuencias observables: un historial anotado con una función de agregación crece y conserva los turnos anteriores, mientras que sin esa anotación cada nodo sobrescribiría el historial y el agente perdería la conversación sin producir ningún error.

Las **aristas** determinan la transición entre nodos. Una arista normal establece una sucesión fija; una **arista condicional** evalúa una función sobre el estado y selecciona el destino. Esta construcción es la que permite expresar el bucle de un agente como parte de la estructura del grafo, en lugar de como un detalle interno de una biblioteca.

La forma más directa de comprobar que el modelo es el descrito es pedirle a la propia biblioteca que dibuje el grafo. La Figura 7 reproduce esa salida para el caso más simple —un único nodo entre el inicio y el fin—: el diagrama no es una ilustración del autor, sino el resultado de serializar la estructura compilada.

5.2 El bucle ReAct como estructura explícita

La materialización del elemento de planificación consiste en construir el ciclo de ReAct [27] con dos nodos y una arista condicional. El primero consulta al modelo; el segundo ejecuta las herramientas solicitadas; y la arista condicional decide si el ciclo continúa o termina. La Figura 8 muestra la estructura resultante.

Dos consecuencias merecen destacarse. La primera es que **el bucle se puede imprimir**: la

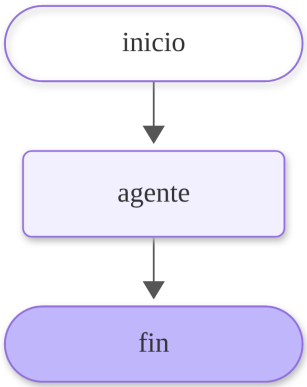


Figura 7: Grafo mínimo generado por la propia biblioteca mediante su método de serialización a Mermaid, y renderizado sin retoques. Salida real de la herramienta.

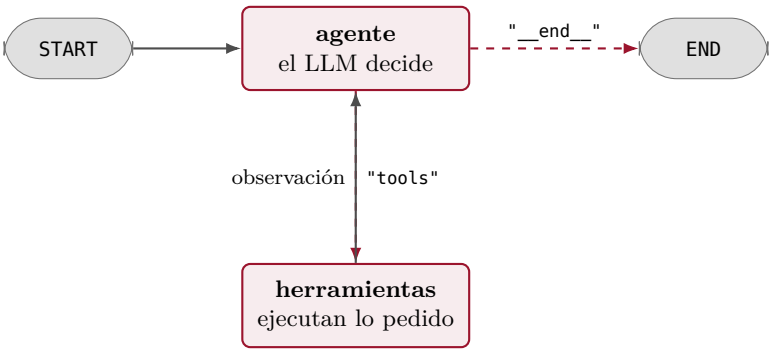


Figura 8: El ciclo ReAct expresado como grafo de estados. La función de decisión inspecciona el último mensaje: si el modelo ha solicitado herramientas, el control pasa al nodo de ejecución; en caso contrario, el grafo finaliza. La arista que retorna del nodo de herramientas al nodo del agente es, literalmente, el bucle de razonamiento y acción.

estructura del agente es un objeto que puede serializarse a un lenguaje de descripción de diagramas y examinarse. La segunda es que los puntos de intervención son evidentes: limitar el número de iteraciones, insertar una validación o añadir una aprobación humana son modificaciones locales sobre una estructura visible.

El diagrama anterior es una reconstrucción anotada, útil para explicar la función de decisión. La Figura 9 muestra, sin anotaciones añadidas, el grafo que la biblioteca genera para esa misma estructura: es el mismo objeto, dibujado por la herramienta a partir de la estructura compilada.

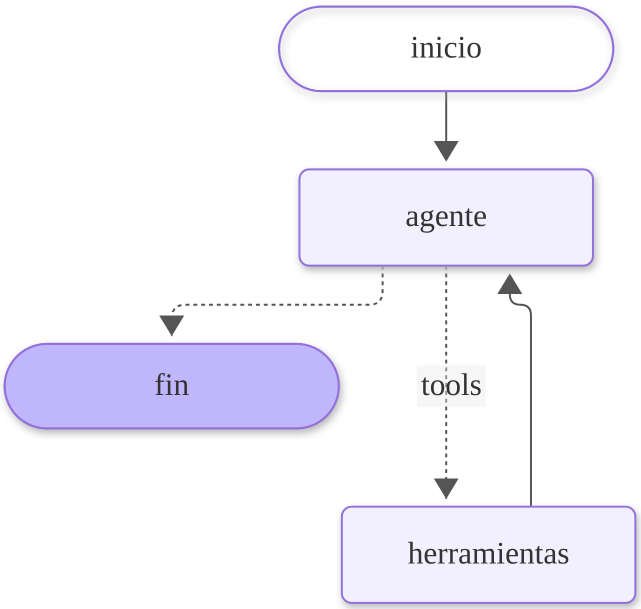


Figura 9: Bucle de razonamiento y acción tal como lo dibuja la propia biblioteca. La etiqueta de la arista condicional procede del valor que devuelve la función de decisión. Salida real de la herramienta.

El mismo formalismo expresa otros flujos de control. La Figura 10 muestra el patrón planificar y ejecutar descrito en la Sección 3.4, en el que un contador del estado gobierna la repetición: la arista etiquetada como «seguir» vuelve al mismo nodo mientras queden pasos. Obsérvese que la diferencia con el bucle anterior no reside en el modelo ni en las herramientas, sino en **dónde vive el plan**: allí en la decisión del modelo, aquí en una clave del estado.

5.3 Persistencia y memoria

La memoria se implementa mediante un mecanismo de **puntos de control** (*checkpointing*). Al compilar un grafo se le puede asociar un almacén de puntos de control; a partir de ese momento, cada ejecución guarda el estado tras cada paso y lo asocia a un identificador de hilo proporcionado en la configuración de la invocación.

La consecuencia es que la memoria a corto plazo deja de ser un añadido y pasa a ser una propiedad del sistema de ejecución: dos invocaciones con el mismo identificador de hilo comparten estado y, por tanto, conversación; dos invocaciones con identificadores distintos están aisladas por construcción. El aislamiento entre conversaciones, que en otros planteamientos requiere lógica adicional, aquí es el comportamiento por defecto.

La memoria a largo plazo se implementa con un **almacén** independiente, organizado en espacios de nombres (*namespaces*) y consultable desde los nodos o desde las herramientas. La

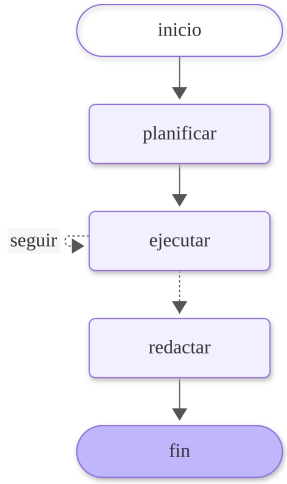


Figura 10: Patrón planificar y ejecutar expresado con el mismo formalismo. La arista de retorno se controla mediante un contador almacenado en el estado, no mediante una decisión del modelo. Salida real de la herramienta.

diferencia con el punto de control es de eje: el punto de control se indexa por conversación, el almacén por usuario. Un agente que debe recordar una preferencia entre conversaciones distintas necesita el segundo; uno que debe recordar lo dicho hace dos turnos necesita el primero.

El compilador admite distintos backends de persistencia —en memoria, en SQLite o en PostgreSQL, entre otros—, de modo que la misma estructura de grafo puede pasar de un prototipo docente a un despliegue con estado duradero sin modificar su lógica. Es importante señalar que los backends en memoria **no sobreviven al proceso**: son adecuados para la enseñanza y para las pruebas, no para producción.

Un aspecto que conviene subrayar por su valor didáctico es que la ausencia de configuración de memoria **no produce ningún error**. El grafo funciona, las respuestas son plausibles y el único síntoma es que el agente no recuerda. Este fallo silencioso es, en la experiencia recogida en el caso práctico, el más costoso de diagnosticar para quien no conoce el mecanismo.

5.4 Interrupciones y supervisión humana

El mecanismo de interrupción permite **pausar la ejecución** en un punto definido por el programador y reanudarla posteriormente con una decisión externa. Su funcionamiento requiere persistencia, porque el estado debe conservarse mientras el grafo está detenido.

La Figura 11 ilustra la secuencia: cada turno genera un punto de control; la interrupción detiene el grafo y expone al exterior el motivo de la pausa; la reanudación se realiza sobre el mismo hilo, con la decisión del supervisor incorporada al estado.

Este mecanismo conecta directamente con el cuarto elemento de la arquitectura: la posibilidad de **aprobar un plan antes de ejecutarlo**. Un agente que elabora un plan explícito puede interrumpirse inmediatamente después de planificar, de modo que la supervisión humana se ejerza sobre el plan y no sobre sus consecuencias. Las guías de interacción humano–IA [35] proporcionan el marco de diseño para decidir en qué puntos debe solicitarse esa intervención.

Tres restricciones del mecanismo tienen consecuencias prácticas que la documentación explicita y que conviene conocer: el nodo que contiene la interrupción **se reejecuta desde el principio** al reanudar, por lo que sus efectos secundarios deben ser idempotentes; la correspondencia entre llamadas y reanudaciones es **posicional**, de modo que no deben introducirse interrupciones en

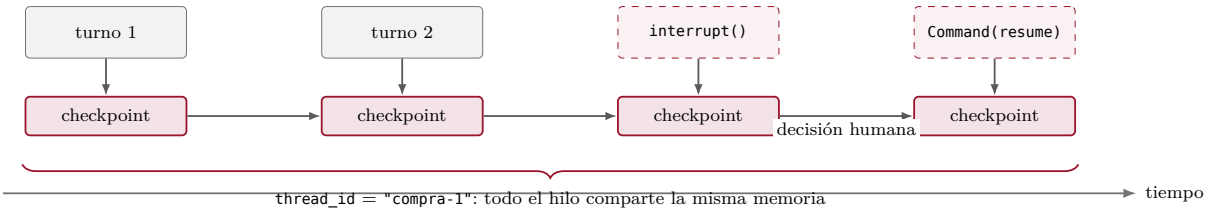


Figura 11: Secuencia de puntos de control con interrupción humana. La ejecución guarda el estado tras cada paso; la interrupción suspende el grafo y publica una pregunta al exterior; la reanudación continúa exactamente donde se detuvo, siempre sobre el mismo identificador de hilo. Elaboración propia.

bucles ni omitirse condicionalmente; y la reanudación debe realizarse siempre sobre el mismo identificador de hilo.

5.5 Transmisión incremental y observabilidad

El marco ofrece dos mecanismos de transmisión incremental. El primero es un parámetro de modo que permite seleccionar qué se emite —el estado completo tras cada paso, las actualizaciones parciales de cada nodo, los fragmentos de texto a medida que el modelo los genera o los eventos asociados a los puntos de control—. El segundo es una interfaz de eventos con proyecciones tipadas que expone de forma separada los mensajes, los valores, los subgrafos y las interrupciones.

La distinción importa por una razón de madurez: la interfaz de eventos es la recomendada en la documentación reciente, pero su versión más nueva está marcada como experimental en el propio código, que emite un aviso al utilizarla. En un contexto docente esto no es un obstáculo, sino un contenido: enseña que las interfaces evolucionan y que conviene distinguir lo estable de lo experimental.

Desde el punto de vista pedagógico, la transmisión incremental cumple además una función motivadora: mejora de forma perceptible la experiencia de uso sin reducir el tiempo real de cómputo. Es una lección útil sobre la diferencia entre latencia percibida y latencia real.

5.6 Patrones multiagente

Sobre el mismo modelo computacional se expresan los patrones multiagente que la literatura ha explorado. La encuesta sobre sistemas multiagente con LLM [37] organiza el campo y describe las topologías habituales. La más frecuente es la de **supervisor**: un agente coordinador que decide a qué especialista corresponde cada subtarea y agrega los resultados. La idea subyace a propuestas como AutoGen [38], que organiza la resolución de tareas como una conversación entre agentes; MetaGPT [39], que codifica flujos de trabajo humanos en procedimientos operativos; CAMEL [40], que estudia la comunicación entre agentes con roles complementarios; y Magentic-One [41], que coordina un equipo generalista con un agente director. En el extremo de la autonomía, Voyager [42] muestra un agente abierto que acumula habilidades en un almacén consultable y las reutiliza.

Desde la perspectiva de este trabajo, la relevancia del patrón supervisor es que **reutiliza el primer elemento de la arquitectura**: un especialista no es más que un perfil con un modelo, unas herramientas y una memoria. El sistema multiagente es, por tanto, el mismo concepto aplicado a escala.

5.7 Comparación con planteamientos alternativos

Conviene cerrar esta sección situando el enfoque en su contexto. Existen otras aproximaciones al desarrollo de agentes, entre ellas los marcos orientados a la conversación entre agentes [38] y los orientados a flujos de trabajo predefinidos. La elección entre ellas depende del grado de autonomía que se desee ceder al modelo.

La posición que se defiende aquí es que un **grafo de estados explícito** es preferible cuando el objetivo es comprender o auditar el comportamiento, por tres razones. Primera: la estructura es un objeto de primera clase, inspeccionable y modificable. Segunda: los puntos de intervención —memoria, supervisión, límites de iteración— son locales y visibles. Y tercera: el mismo formalismo cubre el espectro que va del agente único al sistema multiagente, lo que permite enseñar un solo modelo mental en lugar de varios.

La Tabla 2 sintetiza las capacidades que LangGraph aporta a cada elemento arquitectónico.

Tabla 2: Tabla 2. Correspondencia entre los elementos de la arquitectura y las capacidades del marco. Elaboración propia a partir de [6].

Elemento	Construcción	Capacidad que aporta
Perfil	Nodo con mensaje de sistema	El rol es un parámetro del grafo, sustituible
Herramientas	Nodo de ejecución y arista condicional	El bucle es explícito y acotable
Memoria	Puntos de control y almacén	Continuidad por hilo y persistencia por usuario
Planificación	Aristas condicionales y estado	Estrategias intercambiables sobre el mismo grafo
Supervisión	Interrupciones	Aprobación humana en puntos definidos
Observabilidad	Transmisión incremental y trazas	Inspección de cada paso y de cada invocación

6 Acceso a los modelos: Ollama

6.1 La elección del proveedor como decisión arquitectónica

En las secciones anteriores se ha tratado el modelo como un componente sustituible. Esa sustituibilidad no es neutral: la forma en que se accede al modelo condiciona el coste, la privacidad, la latencia, la reproducibilidad de los materiales y, en el contexto de este trabajo, la viabilidad misma de una sesión docente.

Ollama [43] es una herramienta que permite ejecutar modelos de lenguaje de forma local y, en sus versiones recientes, acceder también a un catálogo de modelos alojados en su infraestructura. Su interés para este trabajo reside en tres propiedades.

La primera es que **expone una interfaz compatible con la API de OpenAI**. Esto significa que un cliente escrito para un proveedor comercial funciona sin modificaciones contra un modelo local, cambiando únicamente la dirección base. Para la docencia, la consecuencia es directa: el código que se enseña no queda atado a un proveedor concreto.

La segunda es que **unifica el acceso local y remoto bajo la misma interfaz**. El mismo cliente puede dirigirse a un modelo que se ejecuta en el equipo del estudiante o a un modelo

alojado, lo que permite diseñar materiales con un mecanismo de respaldo: si el acceso remoto falla o agota su cuota, el mismo código sigue funcionando en local.

La tercera es que **elimina la barrera de entrada económica** para un taller. Un plan gratuito con acceso a un subconjunto de modelos, combinado con la posibilidad de ejecutar modelos pequeños en el propio equipo, permite que un grupo de estudiantes trabaje sin coste y sin depender de una clave de API comercial.

6.2 Despliegue local

En su modalidad local, Ollama se organiza en torno a un **servicio en segundo plano** que escucha en el puerto 11434 de la interfaz de retorno y expone dos superficies de acceso simultáneas: una **API nativa** y un **punto de entrada compatible con la API de OpenAI**. La Figura 12 representa la topología completa.

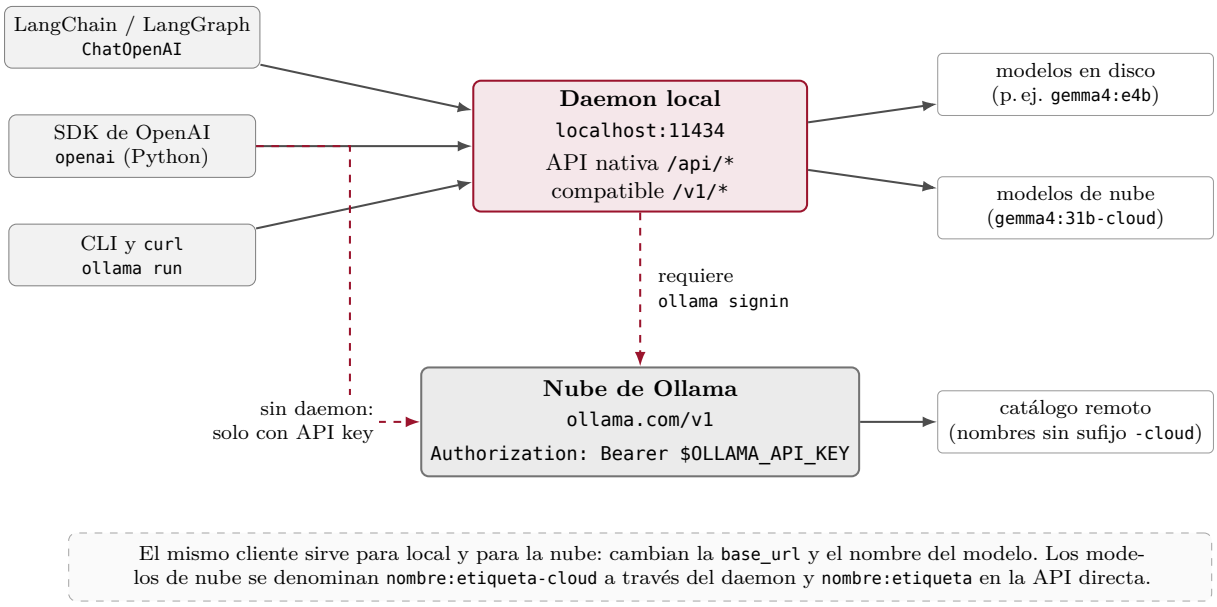


Figura 12: Topología de acceso a modelos con Ollama. Un mismo cliente puede dirigirse al servicio local, que atiende tanto modelos almacenados en disco como modelos remotos invocados a través de él, o directamente al servicio en la nube mediante una clave de API. Los nombres de los modelos difieren entre ambas rutas. Elaboración propia a partir de [@ollama2026].

La API nativa emplea rutas propias para listar modelos, descargarlos, mantener conversaciones y obtener información de capacidades. El punto de entrada compatible con OpenAI ofrece las rutas habituales de este proveedor y es el que utilizan las bibliotecas de cliente estándar. La coexistencia de ambas superficies no es un detalle menor: existen diferencias de comportamiento entre ellas —por ejemplo, en la representación de los argumentos de una invocación de herramienta— que se discuten en la Sección 6.4.

La ejecución local presenta ventajas evidentes de privacidad —los datos no abandonan el equipo— y de disponibilidad —no depende de la red—, y una limitación igualmente evidente: la calidad y el tamaño de los modelos ejecutables están acotados por los recursos de hardware disponibles.

6.3 El servicio en la nube

La modalidad en la nube traslada la inferencia a la infraestructura del proveedor. Existen dos vías de acceso que conviene distinguir con precisión porque se confunden con facilidad.

La primera consiste en **invocar modelos remotos a través del servicio local**. Una vez autenticado el servicio local con la cuenta del usuario, los modelos alojados se direccionan añadiendo un sufijo al nombre del modelo. Esta vía mantiene un único punto de acceso —el servicio local— y no requiere gestionar credenciales en el código.

La segunda consiste en **acceder directamente al servicio en la nube** mediante una clave de API, sin necesidad de tener Ollama instalado. En este caso la dirección base es el punto de entrada remoto y la autenticación se realiza mediante una cabecera de autorización con el esquema de portador.

La diferencia entre ambas rutas tiene una consecuencia práctica que constituye una fuente de error frecuente: **los nombres de los modelos no coinciden**. A través del servicio local, un modelo alojado se denomina con un sufijo que lo identifica como remoto; en el acceso directo, se emplea el identificador del catálogo remoto sin ese sufijo. Un material docente que no explicite esta regla genera confusión desde la primera celda.

6.4 Compatibilidad con la API de OpenAI y llamado a funciones

La compatibilidad con la API de OpenAI es el mecanismo que permite integrar estos modelos con los marcos de desarrollo descritos en las Secciones 4 y 5. Es importante precisar que se trata de una **compatibilidad parcial**: el proveedor implementa un subconjunto de la especificación, y conocer sus límites es requisito para diseñar materiales fiables.

Tabla 3: Tabla 3. Subconjunto de la especificación de OpenAI implementado y sus consecuencias para el diseño de agentes. Elaboración propia a partir de la verificación descrita en la Sección 7.4 y de [43].

Capacidad	Estado	Consecuencia para el diseño de agentes
Conversación (`chat/completions`)	Soportada	Uso general
Llamado a funciones (`tools`)	Soportada	Los agentes con herramientas son viables
Selección forzada de herramienta	No soportada	No puede obligarse al modelo a invocar una función
Salida estructurada por esquema	No soportada en la nube	Debe imponerse por el canal de funciones
Modo JSON	Parcial	No garantiza validez estructural
Respuestas con estado	No soportada en la nube	El estado lo gestiona el orquestador

De esta tabla se derivan tres conclusiones que afectan directamente al diseño de agentes y que el caso práctico explota de forma deliberada.

La primera es que **el llamado a funciones es la capacidad crítica**. Sin él no hay agentes, únicamente generación de texto. Es, por tanto, la comprobación que debe realizarse antes de cualquier otra, y el taller la sitúa como verificación previa obligatoria.

La segunda es que **la ausencia de selección forzada de herramienta** impide garantizar que el modelo invoque una función concreta. Esto refuerza, por una vía distinta, la conclusión de la Sección 3.1: el control sobre el modelo es probabilístico, y la arquitectura debe asumirlo en lugar de presuponer una obediencia que no está garantizada.

La tercera es que **la ausencia de salida estructurada nativa** convierte el canal de funciones en la única vía fiable para obtener datos con formato. Esto convierte una limitación del proveedor en una lección de diseño: cuando el modelo devuelve texto donde se esperaba una estructura, la causa suele ser que se solicitó el formato en lugar de imponerlo.

Existe además una asimetría de comportamiento entre la API nativa y el punto de entrada compatible con OpenAI que conviene documentar. Ante una misma invocación de herramienta, la primera representa los argumentos como un objeto estructurado y el segundo como una cadena de texto con sintaxis JSON. Ambas representaciones son válidas, pero un material docente que no lo advierta produce desconcierto al comparar salidas.

6.5 Reproducibilidad: la caducidad de los catálogos

Un hallazgo empírico obtenido durante la preparación de este trabajo merece un apartado propio, porque afecta a la validez de cualquier material docente basado en servicios de modelos y rara vez se documenta.

Los catálogos de modelos **no son estables**. Durante la verificación realizada, el servicio local enumeraba veinticuatro modelos, de los cuales únicamente ocho respondían correctamente. Los dieciséis restantes fallaban por dos motivos distintos y con consecuencias distintas:

- **Once modelos** devolvieron un error de cuota que indicaba que el modelo no estaba incluido en el plan gratuito. El modelo existe y funciona, pero su uso requiere crédito.
- **Dos modelos** devolvieron un error que indicaba que habían sido **retirados** por el proveedor en una fecha concreta. El modelo ya no existe en el catálogo.
- El resto de los fallos correspondió a modelos que requieren autenticación adicional.

La distinción entre ambas causas es relevante porque exige respuestas distintas: ante un problema de cuota, la solución es cambiar de modelo o asumir un coste; ante un modelo retirado, la única salida es sustituirlo. Un material docente que fije un identificador de modelo en el código queda obsoleto sin previo aviso y falla en el peor momento posible: durante la sesión.

De este hallazgo se derivan dos recomendaciones de diseño que el caso práctico incorpora.

La primera es **no fijar el modelo en el código**, sino resolverlo en tiempo de ejecución mediante una comprobación que verifique que el modelo responde realmente. Es importante subrayar que la comprobación no puede limitarse a consultar el catálogo: la lista incluye modelos inutilizables. La única verificación fiable consiste en **enviar una petición mínima y comprobar la respuesta**.

La segunda es **disponer de un modelo local de respaldo**. Dado que la cuota es una propiedad de la cuenta y no del modelo, cambiar de modelo remoto no resuelve un problema de cuota; la única alternativa real es la ejecución local. La verificación realizada confirma que un modelo local de tamaño reducido soporta el llamado a funciones y puede sostener el taller completo sin conexión.

Finalmente, conviene registrar una limitación operativa del plan gratuito que condiciona el diseño de las demostraciones: el número de peticiones concurrentes es reducido. En consecuencia, las arquitecturas que despliegan varios agentes en paralelo no son viables en ese régimen, y los materiales deben diseñarse para ejecución secuencial.

7 Caso práctico: el taller

7.1 Del fundamento al artefacto docente

El taller que se describe en esta sección es la materialización de los conceptos de las Secciones 3 a 6 en un artefacto verificable. Su diseño responde a una restricción explícita: **cada concepto arquitectónico debe corresponder a una modificación observable del grafo**, no a una explicación teórica. La Figura 13 presenta la progresión.

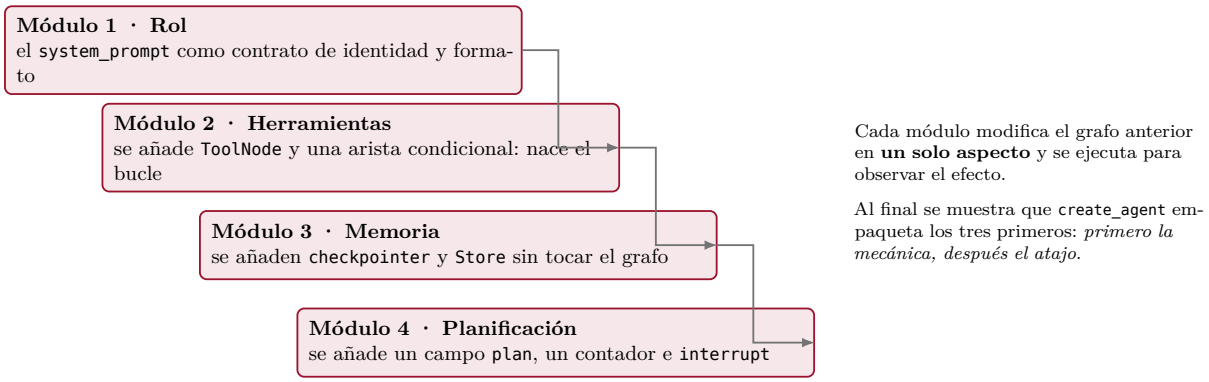


Figura 13: Progresión incremental del taller. Cada módulo añade un único elemento arquitectónico al grafo construido en el módulo anterior, de modo que el efecto de cada adición pueda observarse de forma aislada. Elaboración propia.

El principio didáctico que gobierna la secuencia es el de **divulgación progresiva con verificación inmediata**: el estudiante construye primero el mecanismo y descubre después la abstracción que lo empaqueta. En particular, el bucle de razonamiento y acción se construye manualmente antes de presentar la función que lo genera automáticamente. La justificación es que la comprensión del mecanismo es lo que permite diagnosticar el fallo cuando la abstracción no se comporta como se espera.

7.2 Los cuatro módulos

La Tabla 4 relaciona cada módulo con el elemento arquitectónico que introduce, la construcción concreta que se emplea y la observación que el estudiante debe realizar.

Tabla 4: Tabla 4. Correspondencia entre módulos, elementos arquitectónicos y observaciones. Elaboración propia.

Módulo	Elemento	Construcción	Observación buscada
1	Perfil	Nodo con mensaje de sistema	El mismo grafo con dos perfiles produce respuestas distintas
2	Herramientas	Nodo de ejecución y arista condicional	La traza muestra la petición del modelo y el resultado real
3	Memoria	Puntos de control y almacén	El agente recuerda dentro de un hilo y desconoce otro

Módulo	Elemento	Construcción	Observación buscada
4	Planificación	Estado con plan y contador, interrupción	El plan es legible antes de ejecutarse

El **primer módulo** aborda el perfil y conduce a una conclusión que se refuerza empíricamente: el perfil gobierna el contenido de la respuesta pero no su forma. Se pide al modelo, en tres intentos sucesivos, que devuelva un objeto JSON conforme a un esquema declarado en el mensaje de sistema, y se contabilizan los intentos válidos. El resultado es una tasa de éxito inferior al total, lo que permite introducir la distinción entre solicitar un formato e imponerlo. A continuación se muestra que la imposición por el canal de funciones produce el objeto validado de forma consistente.

El **segundo módulo** introduce las herramientas y constituye el núcleo del taller. El estudiante define funciones decoradas, examina el esquema que el modelo recibe, construye el bucle de forma manual y, finalmente, **lee la traza de la ejecución**. La lectura de la traza se considera el momento didáctico central por una razón concreta: es la única evidencia que permite distinguir un agente que consulta de un generador que alucina.

El módulo cierra mostrando que la función de conveniencia que ofrece el marco construye exactamente el mismo grafo que el alumnado acaba de escribir a mano. La Figura 14 reproduce su estructura interna: los nodos se denominan «model» y «tools» en lugar de «agente» y «herramientas», pero el ciclo es idéntico, y aparecen dos aristas adicionales que en la versión manual no existen y que corresponden a la gestión de errores del modelo y de las herramientas. La comparación cierra el argumento del taller: la abstracción no aporta una estructura distinta, sino una estructura ya escrita y con más casos límite cubiertos.

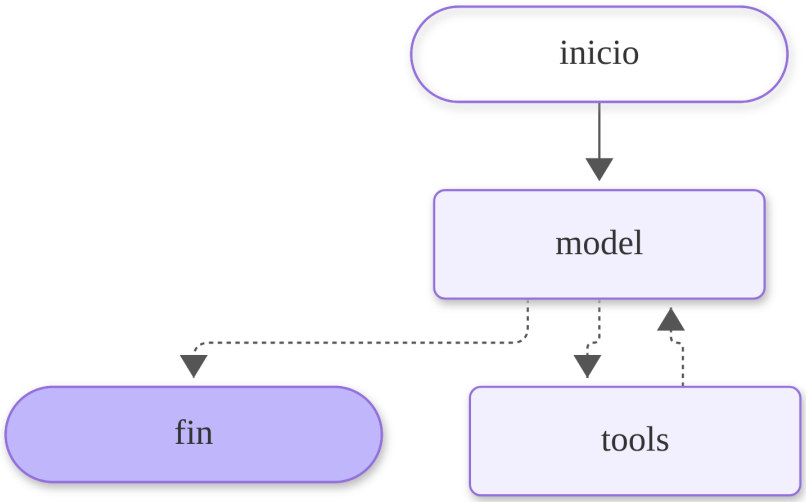


Figura 14: Estructura interna que construye la función de conveniencia del marco, obtenida serializando el grafo compilado. El ciclo entre el modelo y las herramientas es el mismo que el alumnado ha construido manualmente. Salida real de la herramienta.

El **tercer módulo** aborda la memoria en tres actos: sin mecanismo de persistencia el agente no recuerda; con él y con un identificador de hilo constante, recuerda; con un identificador distinto, vuelve a no recordar. Sobre esa base se introduce la memoria a largo plazo y se demuestra con la prueba que da sentido a todo el módulo: **guardar información en una conversación**

y recuperarla desde otra distinta, lo que solo es posible si el agente consulta el almacén mediante una herramienta.

El **cuarto módulo** presenta la planificación. Se contabiliza el número de iteraciones del bucle ReAct para hacer visible su carácter reactivo y se introduce un estado con un plan explícito y un contador. Finalmente se incorpora la interrupción, ejecutando una pausa real y reanudándola con una decisión humana. El módulo cierra el recorrido por los cuatro elementos y deja el patrón completo como ejercicio.

7.3 Metodología de verificación

Un requisito del diseño fue que **ningún material se diera por válido sin haber sido ejecutado**. La verificación se articuló en tres niveles.

El primero es la **ejecución de todas las libretas** que componen el material, de modo que las salidas que recibe el estudiante sean las producidas realmente por el entorno y no ejemplos ilustrativos. La generación de las libretas está automatizada: si una celda falla, la construcción falla.

El segundo es un **procedimiento de verificación del entorno** que comprueba, una por una, las capacidades de las que depende el taller y devuelve un diagnóstico con el motivo de cada fallo. La Tabla 5 recoge las comprobaciones y su resultado en el entorno de referencia.

Tabla 5: Tabla 5. Comprobaciones del procedimiento de verificación y resultado en el entorno de referencia. Elaboración propia.

Comprobación	Resultado
Servicio de modelos accesible	Correcto
Versiones de la pila de desarrollo	Correcto
Modelo disponible y respondiendo	Correcto
Llamado a funciones	Correcto
Salida estructurada por canal de funciones	Correcto
Memoria a corto plazo	Correcto
Memoria a largo plazo	Correcto
Interrupción y reanudación	Correcto
Persistencia en disco	Correcto

El tercero es la **comprobación de capacidades del modelo**, que se realiza antes de cualquier otra actividad y cuya función es detectar de forma temprana la ausencia de soporte para el llamado a funciones. La experiencia recogida justifica su inclusión: un modelo que no soporta esa capacidad produce un agente que responde con plausibilidad pero sin consultar ninguna fuente, un fallo que puede pasar inadvertido durante toda una sesión.

7.4 Resultados

Los resultados que se presentan a continuación no son hipotéticos: proceden de la ejecución registrada durante la verificación del material.

Llamado a funciones. El modelo remoto utilizado en la verificación soportó el llamado a funciones, incluyendo la invocación **simultánea de dos herramientas** en una misma respuesta. Los argumentos se devolvieron correctamente formados. La traza de una ejecución representativa del bucle ReAct fue la siguiente:

```
[00] HumanMessage   ¿Cuánto cuesta el pollo? ¿Y 100 EUR en USD?
[01] AIMessage  ->  TOOL_CALLS: buscar_precio({'producto': 'pollo'}),
                                convertir_moneda({'cantidad': 100, 'destino': 'USD',
                                                  'origen': 'EUR'})
[02] ToolMessage    5.0
[03] ToolMessage    110.0
[04] AIMessage      El pollo cuesta 5.0 EUR y 100 EUR equivalen a 110.0 USD.
```

La lectura de esta traza ilustra el mecanismo descrito en la Sección 3.3: el mensaje [01] no contiene una respuesta sino una **petición**; los mensajes [02] y [03] contienen los valores calculados por el código, no por el modelo; y solo el mensaje [04] es una respuesta construida sobre datos verificados.

Salida estructurada. Se compararon las estrategias de obtención de salida tipada descritas en la Sección 4.3. Los resultados, obtenidos sobre el mismo modelo y la misma petición, se recogen en la Tabla 6.

Tabla 6: Tabla 6. Resultados comparados de las estrategias de salida estructurada. Elaboración propia.

Estrategia	Resultado
Instrucción de formato en el mensaje de sistema	Fallo intermitente
Esquema JSON nativo del proveedor	No soportado
Modo JSON	Fallo: devuelve texto narrativo
Canal de funciones	Correcto y estable

El resultado confirma empíricamente la tesis de la Sección 3.1: la única estrategia fiable es la que impone el formato por un canal distinto del texto. Es también la evidencia que sostiene una de las afirmaciones centrales del taller.

Memoria a largo plazo. La prueba de recuperación entre conversaciones produjo el resultado esperado. Tras guardar una restricción alimentaria en una conversación, una segunda conversación con identificador de hilo distinto recuperó la información mediante una invocación de herramienta, como muestra la traza:

```
[01] AIMessage  ->  TOOL_CALLS: leer_preferencias({})
[02] ToolMessage    {'alergia': 'frutos secos'}
[03] AIMessage      Tienes alergia a los frutos secos. Te propongo una cena ligera...
```

La importancia de esta traza es que demuestra que la información **entró en el contexto en el momento en que se necesitaba**, recuperada desde el almacén, y no porque el sistema reenviara el historial de la conversación anterior, que en este caso no existía.

Planificación. La ejecución del patrón con plan explícito produjo un plan de dos pasos legible antes de su ejecución y una traza de resultados acumulados. El contraste con el bucle reactivo, que requirió dos iteraciones sin conocimiento previo de su número, se utilizó para introducir el compromiso entre ambas estrategias.

7.5 Discusión didáctica

La experiencia de diseño y verificación permite formular cuatro observaciones.

La primera es que **el fallo silencioso es el enemigo principal de la enseñanza de agentes**. Los tres errores más costosos identificados durante la preparación —olvidar la configuración de memoria, no comprobar el soporte de herramientas y solicitar el formato en lugar de imponerlo— comparten una propiedad: no producen error. El sistema funciona y las respuestas son plausibles. Este hallazgo justifica que el material incorpore comprobaciones explícitas y que la lectura de trazas se enseñe como una habilidad y no como un recurso de depuración.

La segunda es que **la distinción entre solicitar e imponer** es probablemente la idea con mayor rendimiento del taller, porque conecta tres conceptos que habitualmente se enseñan por separado: el perfil, la salida estructurada y las herramientas.

La tercera es que la **volatilidad de la infraestructura** debe tratarse como contenido y no como accidente. Un material que fije versiones de modelos quedará obsoleto, y un alumnado que no comprenda por qué falla interpretará el fallo como un defecto del material. Documentar la caducidad de los catálogos y automatizar la verificación convierte un problema en una lección sobre reproducibilidad en sistemas que dependen de servicios externos.

La cuarta es que **la separación entre el modelo y el orquestador debe explicitarse desde el principio**. Buena parte de la confusión inicial del alumnado procede de atribuir al modelo responsabilidades que corresponden al orquestador —la ejecución de herramientas, la persistencia del estado, la selección de lo que se expone— y viceversa.

8 Discusión

8.1 Limitaciones

El trabajo presenta limitaciones que conviene declarar.

La primera es de **alcance empírico**. Los resultados de las Secciones 6.5 y 7.4 corresponden a un único entorno de referencia, con un conjunto concreto de modelos y en una fecha determinada. Dado que el propio trabajo documenta la volatilidad de los catálogos, los resultados deben interpretarse como una caracterización de ese entorno y no como una propiedad estable de la herramienta.

La segunda es de **evaluación del aprendizaje**. El trabajo describe el diseño instruccional y verifica la corrección técnica del material, pero no incluye un estudio controlado con estudiantes que mida su efecto sobre la comprensión. La evaluación de materiales de este tipo requeriría un diseño experimental con grupo de control que queda fuera del alcance de este documento.

La tercera es de **cobertura**. El taller se concentra en los cuatro elementos constitutivos y en un patrón de agente único. Los patrones multiagente se presentan como material complementario y no se evalúan con la misma profundidad.

La cuarta es de **dependencia tecnológica**. Aunque la arquitectura de abstracción de proveedor reduce el acoplamiento, el material emplea interfaces concretas cuya evolución puede invalidar detalles de implementación. La estrategia de verificación automatizada mitiga el riesgo, pero no lo elimina.

8.2 Riesgos y seguridad

La supervisión humana se ha presentado como un elemento de diseño, no como una opción. La justificación académica es sólida: la literatura ha demostrado que un agente que consume contenido externo puede ser manipulado a través de ese contenido [34], lo que convierte cualquier dato recuperado en una entrada potencialmente hostil. Las guías de interacción humano–IA [35] proporcionan los principios de diseño para situar los puntos de control.

En el contexto del taller, esta cuestión se aborda con dos decisiones concretas. La primera es que las herramientas empleadas no tienen efectos sobre el mundo real: son funciones puras sobre datos de ejemplo, lo que elimina el riesgo operativo durante la sesión. La segunda es que todos los ejemplos que producen efectos se someten a una interrupción explícita, de modo que el alumnado interiorice que la autonomía se concede de forma acotada.

Un segundo riesgo, menos evidente, es el **coste y la cuota**. Un agente que itera puede realizar un número elevado de invocaciones al modelo sin que el usuario lo perciba. En un régimen

de cuota limitada, un bucle sin acotar agota el recurso disponible. La existencia de límites de iteración y de contadores explícitos no es, por tanto, una precaución teórica, sino una medida con efectos observables.

8.3 Buenas prácticas

De la experiencia recogida se derivan seis prácticas que se consideran transferibles más allá del caso concreto.

1. **Verificar la capacidad crítica antes de empezar.** Si el modelo no soporta el llamado a funciones, el resto del trabajo carece de sentido. La verificación debe consistir en una petición real, no en la consulta de un catálogo.
2. **No fijar el modelo en el código.** Resolverlo en tiempo de ejecución, con una lista de candidatos y un modelo local de respaldo.
3. **Imponer el formato, no solicitarlo.** Emplear el canal de funciones para la salida estructurada.
4. **Leer la traza como práctica habitual.** La ausencia de invocaciones de herramienta es un síntoma, no un detalle.
5. **Diseñar para el fallo silencioso.** Toda configuración cuya omisión no produzca error —memoria, persistencia, identificadores de hilo— debe comprobarse de forma explícita.
6. **Acotar la autonomía.** Límites de iteración, herramientas sin efectos reales e interrupciones en los puntos de decisión.

9 Conclusiones y trabajo futuro

Este documento ha sintetizado los fundamentos académicos de la arquitectura de agentes basados en modelos de lenguaje, ha identificado los cuatro elementos que la literatura considera constitutivos —perfil, memoria, herramientas y planificación— y ha mostrado su correspondencia con las construcciones de un marco de desarrollo basado en grafos de estado.

La aportación principal no es la enumeración de esos elementos, que la literatura ya proporciona, sino la **articulación entre el fundamento y el artefacto docente**. Cada elemento arquitectónico se ha puesto en correspondencia con una construcción concreta, con un modo de fallo característico y con una observación verificable. Esa triple correspondencia es la que permite convertir una exposición conceptual en un material ejecutable y comprobable.

Del trabajo se desprenden tres conclusiones que se consideran generalizables.

La primera es que **la elección del mecanismo de orquestación es una decisión didáctica además de técnica**. Un formalismo que hace explícita la estructura del agente permite enseñar el mecanismo; uno que la oculta obliga a enseñar la interfaz y deja al alumnado sin recursos para diagnosticar fallos.

La segunda es que **los fallos silenciosos constituyen el contenido más valioso**. Las tres dificultades más costosas identificadas —la memoria no configurada, la ausencia de soporte de herramientas y el formato solicitado en lugar de impuesto— no producen error alguno. Enseñarlas de forma explícita, con la observación que las delata, tiene más valor que presentar el camino que funciona.

La tercera es que **la reproducibilidad de los materiales que dependen de servicios externos exige un tratamiento activo**. La volatilidad de los catálogos de modelos, la variación de las cuotas y la evolución de las interfaces son condiciones del entorno, no accidentes. Automatizar la verificación y documentar la dependencia son prácticas necesarias, no refinamientos.

Como trabajo futuro se identifican tres líneas. La primera es la evaluación controlada del

material con estudiantes, midiendo la capacidad de diagnosticar fallos frente a un grupo que reciba únicamente la interfaz de alto nivel. La segunda es la extensión del caso práctico a los patrones multiagente con la misma metodología de verificación. Y la tercera es el estudio sistemático de la fiabilidad del llamado a funciones en modelos de distinto tamaño, con especial atención a la ejecución local, que es la única alternativa real cuando la cuota del servicio remoto se agota.

Referencias

[1] L. Wang *et al.*, "A survey on large language model based autonomous agents," 2023, Available: https://arxiv.org/abs/2308.11432

[2] Z. Xi *et al.*, "The rise and potential of large language model based agents: A survey," 2023, Available: https://arxiv.org/abs/2309.07864

[3] Y. Cheng *et al.*, "Exploring large language model based intelligent agents: Definitions, methods, and prospects," 2024, Available: https://arxiv.org/abs/2401.03428

[4] G. Mialon *et al.*, "Augmented language models: A survey," 2023, Available: https://arxiv.org/abs/2302.07842

[5] T. R. Sumers, S. Yao, K. Narasimhan, and T. L. Griffiths, "Cognitive architectures for language agents," 2023, Available: https://arxiv.org/abs/2309.02427

[6] *LangGraph: Build resilient agents as graphs.* LangChain, Inc., 2026. Available: https://docs.langchain.com/oss/python/langgraph/overview

[7] A. Vaswani *et al.*, "Attention is all you need," 2017, Available: https://arxiv.org/abs/1706.03762

[8] T. B. Brown *et al.*, "Language models are few-shot learners," 2020, Available: https://arxiv.org/abs/2005.14165

[9] W. X. Zhao *et al.*, "A survey of large language models," 2023, Available: https://arxiv.org/abs/2303.18223

[10] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," 2022, Available: https://arxiv.org/abs/2203.02155

[11] H. W. Chung *et al.*, "Scaling instruction-finetuned language models," 2022, Available: https://arxiv.org/abs/2210.11416

[12] E. Karpas *et al.*, "MRKL systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning," 2022, Available: https://arxiv.org/abs/2205.00445

[13] Y. Ge, Y. Ren, W. Hua, S. Xu, J. Tan, and Y. Zhang, "LLM as OS, agents as apps: Envisioning AIOS, agents and the AIOS-agent ecosystem," 2023, Available: https://arxiv.org/abs/2312.03815

[14] T. Masterman, S. Besen, M. Sawtell, and A. Chao, "The landscape of emerging AI agent architectures for reasoning, planning, and tool calling: A survey," 2024, Available: https://arxiv.org/abs/2404.11584

[15] B. T. Willard and R. Louf, "Efficient guided generation for large language models," 2023, Available: https://arxiv.org/abs/2307.09702

[16] Z. Zhang *et al.*, "A survey on the memory mechanism of large language model based agents," 2024, Available: https://arxiv.org/abs/2404.13501

[17] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," 2023, Available: https://arxiv.org/abs/2304.03442

[18] C. Packer *et al.*, "MemGPT: Towards LLMs as operating systems," 2023, Available: https://arxiv.org/abs/2310.08560

[19] T. Schick *et al.*, "Toolformer: Language models can teach themselves to use tools," 2023, Available: https://arxiv.org/abs/2302.04761

[20] Y. Qin *et al.*, "Tool learning with foundation models," 2023, Available: https://arxiv.org/abs/2304.08354

[21] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive APIs," 2023, Available: https://arxiv.org/abs/2305.15334

[22] Y. Qin *et al.*, "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," 2023, Available: https://arxiv.org/abs/2307.16789

[23] J. Wei *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," 2022, Available: https://arxiv.org/abs/2201.11903

[24] X. Wang *et al.*, "Self-consistency improves chain of thought reasoning in language models," 2022, Available: https://arxiv.org/abs/2203.11171

[25] S. Yao *et al.*, "Tree of thoughts: Deliberate problem solving with large language models," 2023, Available: https://arxiv.org/abs/2305.10601

[26] M. Besta *et al.*, "Graph of thoughts: Solving elaborate problems with large language models," 2023, Available: https://arxiv.org/abs/2308.09687

[27] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," 2022, Available: https://arxiv.org/abs/2210.03629

[28] L. Wang *et al.*, "Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models," 2023, Available: https://arxiv.org/abs/2305.04091

[29] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," 2023, Available: https://arxiv.org/abs/2303.11366

[30] A. Madaan *et al.*, "Self-refine: Iterative refinement with self-feedback," 2023, Available: https://arxiv.org/abs/2303.17651

[31] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, "Improving factuality and reasoning in language models through multiagent debate," 2023, Available: https://arxiv.org/abs/2305.14325

[32] X. Liu *et al.*, "AgentBench: Evaluating LLMs as agents," 2023, Available: https://arxiv.org/abs/2308.03688

[33] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "T-bench: A benchmark for tool-agent-user interaction in real-world domains," 2024, Available: https://arxiv.org/abs/2406.12045

[34] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," 2023, Available: https://arxiv.org/abs/2302.12173

[35] S. Amershi *et al.*, "Guidelines for human-AI interaction," in *Proceedings of the 2019 CHI conference on human factors in computing systems*, 2019. doi: 10.1145/3290605.3300233.

[36] *LangChain: Build context-aware reasoning applications*. LangChain, Inc., 2026. Available: https://docs.langchain.com/

[37] T. Guo *et al.*, "Large language model based multi-agents: A survey of progress and challenges," 2024, Available: https://arxiv.org/abs/2402.01680

[38] Q. Wu *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," 2023, Available: https://arxiv.org/abs/2308.08155

[39] S. Hong *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," 2023, Available: https://arxiv.org/abs/2308.00352

[40] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, "CAMEL: Communicative agents for "mind.[ex]ploration of large language model society," 2023, Available: https://arxiv.org/abs/2303.17760

[41] A. Fourney *et al.*, "Magentic-one: A generalist multi-agent system for solving complex tasks," 2024, Available: https://arxiv.org/abs/2411.04468

[42] G. Wang *et al.*, "Voyager: An open-ended embodied agent with large language models," 2023, Available: https://arxiv.org/abs/2305.16291

[43] *Ollama: Get up and running with large language models*. Ollama, 2026. Available: https://docs.ollama.com/